

Python Application Programming

ANITYA GUPTA

A solid green horizontal bar spanning the width of the slide at the bottom.

Dynamic Attributes – Class

Attributes that are defined at runtime, after creating the objects or instances.

In Python we call all functions, methods also as an object.

So you can define a dynamic instance attribute for nearly anything in Python.

Example

```
class GFG:
    None

def value():
    return 10

# Driver Code
g = GFG()

# Dynamic attribute of a
# class object
g.d1 = value

# Dynamic attribute of a
# function
value.d1 = "Geeks"

print(value.d1)
print(g.d1() == value())
```

Output:

```
Geeks
True
```

- Firstly let's see the objects, g and value(functions are also considered as objects in Python) are the two objects.
- Here the dynamic attribute for both the objects is "d1". This is defined at runtime and not at compile time like static attributes.
- **Note:** The class "GFG" and all other objects or instances of this class do not know the attribute "d1". It is only defined for the instance "g".

Constructors

Special method that is called automatically when an object is created from a class.

Its main role is to initialize the object by setting up its attributes or state.

The method `__new__` is the constructor that creates a new instance of the class while `__init__` is the initializer that sets up the instance's attributes after creation.

These methods work together to manage object creation and initialization.

__new__ Method

This method is responsible for creating a new instance of a class. It allocates memory and returns the new object. It is called before `__init__`

```
class ClassName:
    def __new__(cls, parameters):
        instance = super(ClassName, cls).__new__(cls)
        return instance
```

__init__ Method

This method initializes the newly created instance and is commonly used as a constructor in Python. It is called immediately after the object is created by `__new__` method and is responsible for initializing attributes of the instance.

```
class ClassName:
    def __init__(self, parameters):
        self.attribute = value
```

Note: It is called after `__new__` and does not return anything (it returns **None** by default).

Default Constructor

A **default constructor** does not take any parameters other than **self**. It initializes the object with default attribute values.

```
class Car:
    def __init__(self):

        #Initialize the Car with default attributes
        self.make = "Toyota"
        self.model = "Corolla"
        self.year = 2020

# Creating an instance using the default constructor
car = Car()
print(car.make)
print(car.model)
print(car.year)
```



Output

```
Toyota
Corolla
2020
```

Parameterised Constructor

A **parameterized constructor** accepts arguments to initialize the object's attributes with specific values.

```
class Car:
    def __init__(self, make, model, year):

        #Initialize the Car with specific attributes.
        self.make = make
        self.model = model
        self.year = year

# Creating an instance using the parameterized constructor
car = Car("Honda", "Civic", 2022)
print(car.make)
print(car.model)
print(car.year)
```

Output

```
Honda
Civic
2022
```

Encapsulation

Encapsulation is the process of hiding the internal state of an object and requiring all interactions to be performed through an object's methods. This approach:

- Provides better control over data.
- Prevents accidental modification of data.
- Promotes modular programming.

Python achieves encapsulation through public, protected and private attributes.

Working of Encapsulation

Data Hiding: The variables (attributes) are kept private or protected, meaning they are not accessible directly from outside the class. Instead, they can only be accessed or modified through the methods.

Access through Methods: Methods act as the interface through which external code interacts with the data stored in the variables. For instance, getters and setters are common methods used to retrieve and update the value of a private variable.

Control and Security: By encapsulating the variables and only allowing their manipulation via methods, the class can enforce rules on how the variables are accessed or modified, thus maintaining control and security over the data

Public Members

Public members are accessible from anywhere, both inside and outside the class. These are the default members in Python.

Example:

```
class Public:
    def __init__(self):
        self.name = "John" # Public attribute

    def display_name(self):
        print(self.name) # Public method

obj = Public()
obj.display_name() # Accessible
print(obj.name) # Accessible
```

Explanation:

- **Public Attribute (name):** This attribute is declared without any underscore prefixes. It is accessible from anywhere, both inside and outside of the class.
- **Public Method (display_name):** This method is also accessible from any part of the code. It directly accesses the public attribute and prints its value.
- **Object (obj):** An instance of Public is created, and the display_name method is called, demonstrating how public attributes and methods can be accessed directly.

Note: The `__init__` method is a constructor and runs as soon as an object of a class is instantiated.

Protected Members

Protected members are identified with a single underscore (`_`). They are meant to be accessed only within the class or its subclasses.

Example:

```
class Protected:
    def __init__(self):
        self._age = 30 # Protected attribute

class Subclass(Protected):
    def display_age(self):
        print(self._age) # Accessible in subclass

obj = Subclass()
obj.display_age()
```

Explanation:

- **Protected Attribute (`_age`):** This attribute is prefixed with a single underscore, which by convention, suggests that it should be treated as a protected member. It's not enforced by Python but indicates that it should not be accessed outside of this class and its subclasses.
- **Subclass:** Here, a subclass inherits from `Protected`. Within this subclass, we can still access the protected **attribute** `_age`.
- **Method (`display_age`):** This method within the subclass accesses the protected attribute and prints its value. This shows that protected members can be accessed within the class and its subclasses.

Private Members

Private members are identified with a double underscore (`__`) and cannot be accessed directly from outside the class. Python uses name mangling to make private members inaccessible by renaming them internally.

Note: Python's private and protected members can be accessed outside the class through [python name mangling](#).

```
class Private:
    def __init__(self):
        self.__salary = 50000 # Private attribute

    def salary(self):
        return self.__salary # Access through public method

obj = Private()
print(obj.salary()) # Works
#print(obj.__salary) # Raises AttributeError
```

Explanation:

- **Private Attribute (`__salary`):** This attribute is prefixed with two underscores, which makes it a private member. Python enforces privacy by name mangling, which means it renames the attribute in a way that makes it hard to access from outside the class.
- **Method (`salary`):** This public method provides the only way to access the private attribute from outside the class. It safely returns the value of `__salary`.
- **Direct Access Attempt:** Trying to access the private attribute directly (`obj.__salary`) will result in an `AttributeError`, showing that direct access is blocked. This is Python's way of enforcing encapsulation at a language level.